JÖNKÖPING UNIVERSITY
School of Engineering

TARI29 Artificial Intelligence, 7.5 credits, Autumn 2023

Exam 3

Vladimir Tarasov and Alexandros Tzanetos

Date: 5 June 2024 14:00–19:00

Allowed aids:

- VS Code and SWI-Prolog installed at the computers in the exam room
- SWI-Prolog documentation provided as PDF
- The logical equivalence sheet provided as PDF
- Other material provided as PDF

However, you are not allowed to use any other form of assistance. Specifically, it is of course strictly prohibited to collaborate with other students or use online resources during the exam.

Instructions for answering questions:

- Each question will provide a task description and the maximum amount of points that can be awarded for a correct answer.
- Type in your answers directly in the Inspera system.
- If a question mentions that you can submit a drawing or other type of writing on paper, you can get paper sheets from the personnel in the exam room. Then you need to hand in your sheets to the personnel. The paper sheets will be delivered to the examiners after the exam. The paper sheets will be delivered to the examiners after the exam. If you write on paper, **write in legible (clear) handwriting** so that your answer is well readable.
- For Prolog problems, write and debug your code in the VS Code environment. Then copy the Prolog code of your solution from VS Code and paste it directly in the text field of the corresponding question.

Grading:
The maximum total score for the written examination is 70 points, and the grading is:

3	28–41.5	points
4	42–55.5	points
5	56–70	points

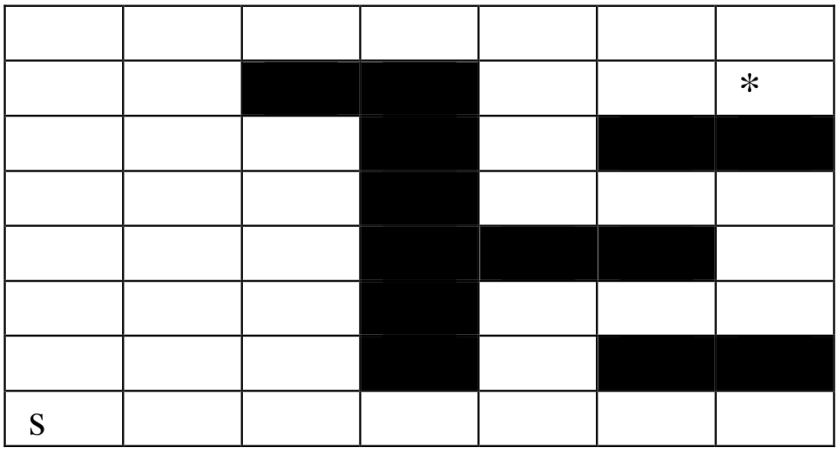
The distribution of points among the exam sections:

Section:	Search	Game Theory	MDP	Logic	Prolog	Total
Points:	12	12	9	14	23	70

A Search (12 points)

1. (3 points) **Search: A*–Task 1**.

Have a look at the very simple world below. The agent is located in the cell marked S and the goal is the cell marked *. The agent can only move horizontal or vertical, i.e., not diagonal, and we want to find the shortest possible path. All black cells are blocked, i.e., cannot be entered by the agent.



We are going to use A* to find the optimal path:

- Suggest a good heuristic function for this problem.

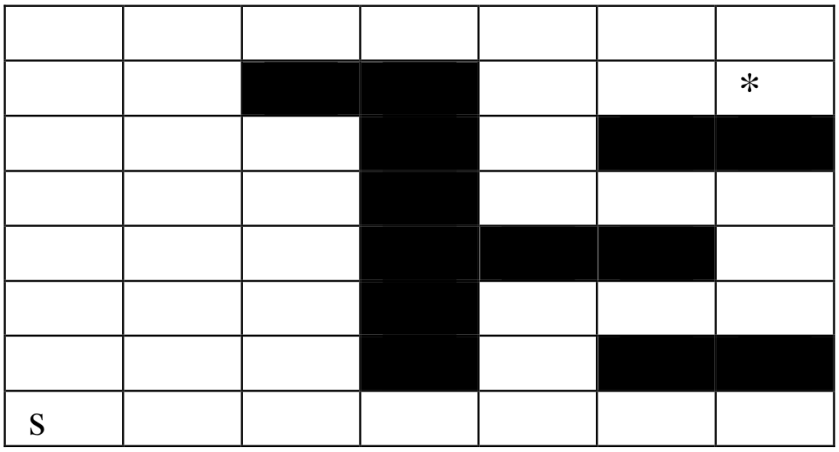
Solution and Marking:

Here, the motivation is the key part to score the points.

Manhattan distance, i.e., disregarding blocked cells.

2. (3 points) **Search: A*–Task 2**.

Have a look at the very simple world below. The agent is located in the cell marked S and the goal is the cell marked *. The agent can only move horizontal or vertical, i.e., not diagonal, and we want to find the shortest possible path. All black cells are blocked, i.e., cannot be entered by the agent.



We are going to use A* to find the optimal path:

- Argue for why the function you suggested in question 1 is a good heuristic function, i.e., that it is admissible, and more informed than at least one other alternative.

Solution and Marking:

Here, the motivation is the key part to score the points.

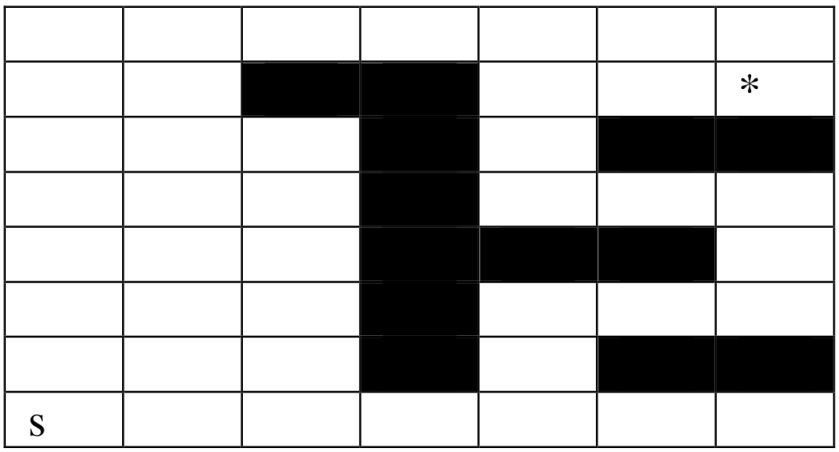
It is clearly admissible, and it is more informed than for instance Euclidean distance.

Marking:

- -2p if only a definition of admissible function is mentioned but no argument for the suggested function

3. (6 points) **Search: A*–Task 3**.

Have a look at the very simple world below. The agent is located in the cell marked S and the goal is the cell marked *. The agent can only move horizontal or vertical, i.e., not diagonal, and we want to find the shortest possible path. All black cells are blocked, i.e., cannot be entered by the agent.



We are going to use A* to find the optimal path:

- Show the order in which the different states are explored by writing numbers in the cells, starting with a 0 in position s and a 1 in the next explored state, etc. Please note the following rules: if there are several states with equal values, the agent first explores the last state found. (The open list is a stack.) If this is not enough to break the tie, the order is E, S, W, N.

Solution and Marking:

In each cell, the first digit shows the order of expansion, the number inside the parentheses is f(x) for that cell.

	24(14)	25(14)	26(14)	27(14)	28(14)	29(14)
	23(12)					30(14)*
	22(12)	17(12)				
	21(12)	16(12)				12(12)
	20(12)	15(12)				11(12)
	19(12)	14(12)		8(12)	9(12)	10(12)
	18(12)	13(12)		7(12)		
0(12)s	1(12)	2(12)	3(12)	4(12)	5(12)	6(12)

Marking:

- −2p if an unneeded path is added
- 0p If no path is given

Marking:

B Game Theory (12 points)

4. (5 points) **Game Theory: Task 1**

Consider the payoff matrix below for a game in normal form.

		Player B		
		X	**Y**	**Z**
	X	**2**,**0**	**1**,**1**	**4**,**2**
Player A	**Y**	**3**,**4**	**1**,**2**	**2**,**3**
	Z	**1**,**3**	**0**,**2**	**3**,**0**

Are there any pure Nash Equilibria or Pareto Optimal outcomes in the game? If so, what are they (and why)?

Solution and Marking:

In a Nash Equilibrium, no player wants to change to another possible action since the payoff will be smaller.

(**Y=3**,**X=4**) is a Nash Equilibrium since **Player A** doesn't want to choose action **X=2** or **Z=1**, since action **Y=3** yields a higher payoff, and **Player B** doesn't want to choose action **Y=2** or **Z=3**, since action **X=4** yields a higher payoff.

The same applies to (**X=4**,**Z=2**). **Player A** doesn't want to choose action **Y=2** or **Z=3**, since action **X=4** yields a higher payoff, and **Player B** doesn't want to choose action **X=0** or **Y=1**, since action **Z=2** yields a higher payoff. Therefore, (**X=4**,**Z=2**) is a Nash Equilibrium, as well.

Pareto optimality is defined as:

- If an outcome o is at least as good for each agent as another outcome $o^{'}$, where at least one agent strictly prefers o over $o^{'}$, then o Pareto dominates $o^{'}$.
- An outcome $o*$ is Pareto optimal if there isn't any other outcome that Pareto dominates $o*$.

Both (**Y=3**,**X=4**) and (**X=4**,**Z=2**) dominate the other outcomes because at least one agent strictly prefers one of those outcomes compared to any of the rest. Moreover, both (**Y=3**,**X=4**) and (**X=4**,**Z=2**) are Pareto Optimal outcomes since it Pareto dominate all other outcomes, i.e., no player would like to change their action in any of these outcomes.

Marking:

- -1p for each incorrect or missing Nash Equilibria. ($\times 2$)
- -0.5p for insufficient justification of Nash Equilibria. ($\times 2$)
- -0.5p for each incorrect or missing Pareto Optimal outcome. ($\times 2$)
- -0.5p for insufficient justification of Pareto Optimal outcomes. ($\times 2$)

5. (3 points) **Game Theory: Task 2**

Let us assume the following scenario. Two pharmaceutical laboratories, SwedPharm and NordLab, want to develop a new drug. They can either team up and aim to develop a new generation drug, or each can work on its own to develop a less ambitious drug. Based on their decision, the following cases may occur:

- if they team up and agree to develop the new generation drug together, then a better product will be released on the market, leading to profits of 2 billion SEK for each company
- if they defect and work on their own project, they will develop a less ambitious drug, leading to lower profits for both, equal to 1 billion SEK
- but if one cooperates (i.e., starts working on an agreed common project) and the other defects (i.e., even though they have agreed on a common project, this company starts working on their own), then the common project will fail (i.e., no profits) while the latter develops a less ambitious drug and earns a profit of 1 billion SEK.

Represent the above situation as a game in its Normal Form. What is the dominant strategy of each company, and why? Can we predict what the outcome of this game will be? Do we make any assumptions to make this prediction?

Solution and Marking:

We represent the game in its Normal Form, as shown in the payoff matrix below:

		NordLab	
		Cooperate	**Defect**
SwedPharm	**Cooperate**	**2**,**2**	**0**,**1**
	Defect	**1**,**0**	**1**,**1**

About dominance, we know the following:

- A **dominant strategy** is a strategy that **dominates all other strategies**.
- A strategy profile that consists of **dominant strategies for each player** must necessarily constitute a **Nash Equilibrium** (no player wants to change strategy). Two types of dominance exist:
  - **Strict dominance:** For every strategy that the other players can play, if the utility of strategy s_i for player i is larger than any other strategy s'_i for player i, then there is a strict dominance of strategy s_i over strategy s'_i.
  - **Weak dominance:** The same as the above, but strategy s_i is larger or equal to strategy s'_i, which means that there is a weak dominance of strategy s_i over strategy s'_i.
- An **equilibrium that only consists of strict dominant strategies** must be **unique**.

For **SwedPharm**, **no strategy** is strictly dominated by the other strategy, since:

- if $a_{NordLab} = Cooperate$ then $a^*_{SwedPharm} = Cooperate$, where $u_{SwedPharm}(Defect, Cooperate) = 1 < u_{SwedPharm}(Cooperate, Cooperate) = 2$.
- if $a_{NordLab} = Defect$ then $a^*_{SwedPharm} = Defect$, where $u_{SwedPharm}(Defect, Defect) = 1 > u_{SwedPharm}(Cooperate, Defect) = 0$.

For **NordLab**, **no strategy** is strictly dominated by the other strategy, since:

- if $a_{SwedPharm} = Cooperate$ then $a^*_{NordLab} = Cooperate$, where $u_{NordLab}(Cooperate, Defect) = 1 < u_{NordLab}(Cooperate, Cooperate) = 2$.
- if $a_{SwedPharm} = Defect$ then $a^*_{NordLab} = Defect$, where $u_{NordLab}(Defect, Defect) = 1 > u_{NordLab}(Defect, Cooperate) = 0$.

Therefore, no dominant strategy exist for either **SwedPharm** or **NordLab**. However, we notice that two Nash Equilibria exist, i.e., (**Cooperate**,**Cooperate**) and (**Defect**,**Defect**).

Considering that for an outcome that constitutes a **Nash Equilibrium**, **no agent can increase its utility** by changing to another action, we predict that the game's outcome will be either (**Cooperate**,**Cooperate**) or (**Defect**,**Defect**). We can assume that the companies will aim for the Pareto efficient outcome, i.e., (**Cooperate**,**Cooperate**) which Pareto dominates (**Defect**,**Defect**).

To predict that outcome, we assumed that **both companies** would act **rationally**.

Marking:

- -0.5p for incorrect or missing Normal Form.
- -0.5p for each incorrect or missing dominant strategy. ($\times 2$)
- -0.5p for the incorrect or missing outcome of the game. ($\times 2$)
- -0.5p for incorrect or missing assumptions.

6. (4 points) **Game Theory: Task 3**

Calculate a Nash Equilibrium with mixed strategies from the payoff matrix below and state the value of the game at this equilibrium. Use the method of assuming the opponent should be indifferent between its actions. Show all your calculations.

		Player B	
		X	**Y**
Player A	**X**	**-1**,**-1**	**4**,**0**
	Y	**0**,**4**	**1**,**1**

Solution and Marking:

We can denote the probability that **Player A** plays strategy **X** with **p** (and **1-p** for strategy **Y**) and that **Player B** plays strategy **X** with probability **q** (and **1-q** for strategy **Y**).

			Player B	
			q	**1-q**
			X	**Y**
Player A	**p**	**X**	**-1**,**-1**	**4**,**0**
	1-p	**Y**	**0**,**4**	**1**,**1**

We then reason that **Player B** has chosen his or her probabilities **q** and **1-q** in such a way that **Player A** is indifferent between strategy **X** and **Y**. Likewise, we reason that **Player A** has chosen his or her probabilities **p** and **1-p** in such a way that **Player B** is indifferent between strategy **X** and **Y**. This gives us the equations below:

Player A (row X = row Y)

$$\begin{aligned} -1q + 4(1-q) &= 0q + 1(1-q) \\ -q + 4 - 4q &= 1 - q \\ -q + q - 4q &= -4 + 1 \\ -4q &= -3 \\ q &= 3/4 \\ 1 - q &= 1/4 \end{aligned}$$

Player B (column X = column Y)

$$\begin{aligned} -1p + 4(1-p) &= 0p + 1(1-p) \\ -p + 4 - 4p &= 1 - p \\ -p + p - 4p &= 1 - 4 \\ -4p &= -3 \\ p &= 3/4 \\ 1 - p &= 1/4 \end{aligned}$$

This gives us q = 3/4, 1-q = 1/4 and p = 3/4, 1-p = 1/4. Hence, we have a Nash Equilibrium with the mixed strategies:

- Player A plays strategy X 1/2 of the time and strategy Y 1/2 of the time.
- Player B plays strategy X 1/4 of the time and strategy Y 3/4 of the time.

The value of the game at this mixed-strategy Nash Equilibrium is obtained by multiplying each pay-off for each player in each outcome (cell) with the corresponding probability for Player A and Player B in that outcome, and finally summing the result in each cell (i.e., the expected value):

Player A

$$\begin{aligned}
U_A &= -1pq + 4p(1-q) + 0(1-p)q + 1(1-p)(1-q) \\
&= -1pq + 4p - 4pq + 0q - 0pq + 1 - q - p + pq \\
&= -4pq + 3p - q + 1 \\
&= -4 \times 3/4 \times 3/4 + 3 \times 3/4 - 3/4 + 1 \\
&= -9/4 + 9/4 - 3/4 + 1 \\
&= -3/4 + 4/4 \\
&= 1/4 \\
&\approx 0.25
\end{aligned}$$

Player B

$$\begin{aligned}
U_B &= -1pq + 4p(1-q) + 0(1-p)q + 1(1-p)(1-q) \\
&= -pq + 4p - 4pq + 0q - 0pq + 1 - q - p + pq \\
&= -4pq + 3p - q + 1 \\
&= -4 \times 3/4 \times 3/4 + 3 \times 3/4 - 3/4 + 1 \\
&= -9/4 + 9/4 - 3/4 + 1 \\
&= -3/4 + 4/4 \\
&= 1/4 \\
&\approx 0.25
\end{aligned}$$

The value of the game is, hence: $(U_A, U_B) = (1/4, 1/4) \approx (0.25, 0.25)$.

Marking:

- -1p for an incorrect mixed strategy per player. (Mixed strategies: 2p)
- -1p for an incorrect game value per player. (Value of the game: 2p)

C MDP / Reinforcement Learning (9 points)

7. (3 points) **MDP / Reinforcement Learning: Task 1**

Assume an MDP consists of the two states S_1 and S_2, the two actions A_1 and A_2, a discount factor $\gamma = 1/2$, a reward function R and a transition function T as shown below:

$$R(s,a,s') = \begin{cases} 1/2 & \text{if } s' = S_1 \\ -1/2 & \text{if } s' = S_2 \end{cases}$$

s	a	s'	T(s,a,s')
S_1	A_1	S_1	3/4
S_1	A_1	S_2	1/4
S_1	A_2	S_1	1/2
S_1	A_2	S_2	1/2
S_2	A_1	S_1	1/3
S_2	A_1	S_2	2/3
S_2	A_2	S_1	1/2
S_2	A_2	S_2	1/2

Perform one iteration of value iteration and enter the value for $V_1(S_1)$ in the table below (you don't have to enter a value into the gray cell). Use the initial values for $V_0(s)$ below instead of initializing all values to 0. Show all your calculations.

s	$V_0(s)$	$V_1(s)$
S_1	4	?
S_2	2	

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s')\right]$$

Solution and Marking:

By substituting values into the value iteration update equation, we get:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s')\right]$$

$$V_1(S_1) \leftarrow max \begin{pmatrix} A_1 : T(S_1,A_1,S_1)\left[R(S_1,A_1,S_1) + \gamma V_0(S_1)\right] + T(S_1,A_1,S_2)\left[R(S_1,A_1,S_2) + \gamma V_0(S_2)\right] \\ A_2 : T(S_1,A_2,S_1)\left[R(S_1,A_2,S_1) + \gamma V_0(S_1)\right] + T(S_1,A_2,S_2)\left[R(S_1,A_2,S_2) + \gamma V_0(S_2)\right] \end{pmatrix}$$

$$V_1(S_1) \leftarrow max \begin{pmatrix} A_1 : \frac{3}{4} * \left[\frac{1}{2} + \frac{1}{2} * 4\right] + \frac{1}{4} * \left[-\frac{1}{2} + \frac{1}{2} * 2\right] = \frac{3}{4} * (\frac{1}{2} + 2) + \frac{1}{4} * (-\frac{1}{2} + \frac{2}{2}) = \frac{3}{4} * \frac{5}{2} + \frac{1}{4} * \frac{1}{2} \\ = \frac{15}{8} + \frac{1}{8} = \frac{16}{8} = 2 \\ A_2 : \frac{1}{2} * \left[\frac{1}{2} + \frac{1}{2} * 4\right] + \frac{1}{2} * \left[-\frac{1}{2} + \frac{1}{2} * 2\right] = \frac{1}{2} * (\frac{1}{2} + 2) + \frac{1}{2} * (-\frac{1}{2} + \frac{2}{2}) = \frac{1}{2} * \frac{5}{2} + \frac{1}{2} * \frac{1}{2} \\ = \frac{5}{4} + \frac{1}{4} = \frac{6}{4} = \frac{3}{2} = 1.5 \end{pmatrix}$$

s	$V_0(s)$	$V_1(s)$
S_1	4	2
S_2	2	

Marking:

- -2.5p for mistakes in the formula. (2p+0.5p)

- -0.125p for each incorrect or missing parameter in the equation. ($16 \times 0.125 = 2$)
- -0.5p for **any** calculation errors.

- -0.5p for the wrong result (of $V_1(s)$).

8. (6 points) **MDP / Reinforcement Learning: Task 2**

Assume an MDP consists of the two states S_1 and S_2, the two actions A_1 and A_2, and a transition function T as shown below:

s	a	s'	T(s,a,s')
S_1	A_1	S_1	1/4
S_1	A_1	S_2	3/4
S_1	A_2	S_1	1/2
S_1	A_2	S_2	1/2
S_2	A_1	S_1	1/4
S_2	A_1	S_2	3/4
S_2	A_2	S_1	1/2
S_2	A_2	S_2	1/2

Assume Q-learning with a learning rate $\alpha = 1/4$ and without discount (i.e., $\gamma = 1$) is used. The following episode is observed (i.e., first, the agent executes the action A_1 in state S_1, ending up in state S_1 with the reward *-1*, followed by the agent executing the action A_2 in state S_1, ending up in state S_2 with the reward *1*):

s_1	a_1	r_1	s_2	a_2	r_2	s_3
S_1	A_1	-1	S_1	A_2	1	S_2

Use the initial Q-values Q_0 in the table below, and enter the values for Q_1 and Q_2. Show all your calculations.

s	a	$Q_0(s,a)$	$Q_1(s,a)$	$Q_2(s,a)$
S_1	A_1	1/2	?	?
S_1	A_2	1/8	?	?
S_2	A_1	5/4	?	?
S_2	A_2	1/4	?	?

$$Q_{k+1}(s,a) \leftarrow Q_k(s,a) + \alpha \left[r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a) \right]$$

Solution and Marking:

By substituting values into the q-value update equation we get:

$$Q_{k+1}(s,a) \leftarrow Q_k(s,a) + \alpha \left[r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a) \right]$$

First step:

$$Q_1(S_1, A_1) \leftarrow Q_0(S_1, A_1) + \frac{1}{4}\left[-1 + 1 \cdot \max_{a'} Q_0(S_1, a') - Q_0(S_1, A_1)\right]$$
$$\leftarrow \frac{1}{2} + \frac{1}{4}\left[-1 + 1 \cdot \max(\frac{1}{2}, \frac{1}{8}) - (\frac{1}{2})\right] = \frac{1}{2} + \frac{1}{4}\left[-1 + \frac{1}{2} - \frac{1}{2}\right] = \frac{1}{2} - \frac{1}{4} = \frac{1}{4}$$

$Q_1(S_1, A_2) \leftarrow Q_0(S_1, A_2)$ since A_2 was not taken in S_1 during step 1 in the episode
$Q_1(S_2, A_1) \leftarrow Q_0(S_2, A_1)$ since A_1 was not taken in S_2 during step 1 in the episode
$Q_1(S_2, A_2) \leftarrow Q_0(S_2, A_2)$ since A_2 was not taken in S_2 during step 1 in the episode

Second step:

$Q_2(S_1, A_1) \leftarrow Q_1(S_1, A_1)$ since A_1 was not taken in S_1 during step 2 in the episode

$$Q_2(S_1, A_2) \leftarrow Q_1(S_1, A_2) + \frac{1}{4}\left[1 + 1 \cdot \max_{a'} Q_1(S_2, a') - Q_1(S_1, A_2)\right]$$
$$\leftarrow \frac{1}{8} + \frac{1}{4}\left[1 + 1 \cdot \max(\frac{5}{4}, \frac{1}{4}) - \frac{1}{8}\right] = \frac{1}{8} + \frac{1}{4}\left[1 + \frac{5}{4} - \frac{1}{8}\right]$$
$$= \frac{1}{8} + \frac{1}{4}\left[1 + \frac{10}{8} - \frac{1}{8}\right] = \frac{1}{8} + \frac{1}{4}\left[1 + \frac{9}{8}\right] = \frac{1}{8} + \frac{1}{4}\left[\frac{8}{8} + \frac{9}{8}\right] = \frac{1}{8} + \frac{1}{4} \cdot \frac{17}{8}$$
$$= \frac{1}{8} + \frac{17}{32} = \frac{4}{32} + \frac{17}{32} = \frac{21}{32}$$

$Q_2(S_2, A_1) \leftarrow Q_1(S_2, A_1)$ since A_1 was not taken in S_2 during step 2 in the episode
$Q_2(S_2, A_2) \leftarrow Q_1(S_2, A_2)$ since A_2 was not taken in S_2 during step 2 in the episode

s	a	$Q_0(s,a)$	$Q_1(s,a)$	$Q_2(s,a)$
S_1	A_1	1/2	1/4	1/4
S_1	A_2	1/8	1/8	21/32
S_2	A_1	5/4	5/4	5/4
S_2	A_2	1/4	1/4	1/4

Marking:

- -2p for mistakes in the formula of the first step. (1.4p+0.6p)
  - -0.2p for each incorrect or missing parameter in the equation. ($7 \times 0.2 = 1.4$)
  - -0.6p for **any** calculation errors.
- -2p for mistakes in the formula of the second step. (1.4p+0.6p)
  - -0.2p for each incorrect or missing parameter in the equation. ($7 \times 0.2 = 1.4$)
  - -0.6p for **any** calculation errors.
- -1p for calculating anything apart from $Q_1(S_1, A_1)$ in the first step.
- -1p for calculating anything apart from $Q_2(S_1, A_1)$ in the second step.

D Logic (14 points)

9. (4 points) **Logic: First-order logic—Task 1**

Consider the following knowledge base, given in natural language:

```
William was king.
All kings have at least one ruthless son.
All sons of kings are princes.
William Rufus and Henry were sons of William.
Henry was not ruthless, but William Rufus was.
All ruthless princes become kings.
No king who is ruthless or crazy is good.
```

Translate the knowledge base into first-order logic, using one FOL sentence for each sentence above.

You must use the following predicates and constants:

- *Unary predicates*; i.e. taking one argument: king, ruthless, prince, crazy, good.
- *Binary predicates*; i.e. taking two arguments: son.
- *Constants*; i.e. names of people: William, Henry, William Rufus (please note that "William Rufus" is a single name referring to one person).

Solution and Marking:

Suggested translations into FOL:

1. king(William)
2. $\forall x\ (\text{king}(x) \Rightarrow \exists y\ (\text{son}(y, x) \wedge \text{ruthless}(y))$
3. $\forall x\ \forall y\ (\text{king}(y) \wedge \text{son}(x, y) \Rightarrow \text{prince}(x))$
4. son(William Rufus, William) $\wedge$ son(Henry, William)
5. $\neg$ruthless(Henry) $\wedge$ ruthless(William Rufus)
6. $\forall x\ (\text{ruthless}(x) \wedge \text{prince}(x) \Rightarrow \text{king}(x))$
7. $\forall x\ (\text{king}(x) \wedge (\text{crazy}(x) \vee \text{ruthless}(x)) \Rightarrow \neg\text{good}(x))$

Marking:

- 0.5p per a right sentence

10. (5 points) **Logic: First-order logic—Task 2**

Consider the following knowledge base, given in natural language:

```
William was king.
All kings have at least one ruthless son.
All sons of kings are princes.
```

```
William Rufus and Henry were sons of William.
Henry was not ruthless, but William Rufus was.
All ruthless princes become kings.
No king who is ruthless or crazy is good.
```

Use the translation of this knowledge base into first-order logic that you did in the question "Logic: First-order logic—Task 1".

We now want to prove that the assertion "**William Rufus was king, but not good**" follows from the knowledge base above. Perform this proof by *reasoning about the formalized sentences above*, taking the semantics of quantifiers and connectives into account.

Solution and Marking:

The sentence to be proved is, in FOL, king(William Rufus) $\wedge$ $\neg$good(William Rufus), which is a conjunction and most easily proved by proving the two separate parts.

From king(William), son(William Rufus, William) and $\forall x\, \forall y\, (\text{king}(y) \wedge \text{son}(x, y) \Rightarrow \text{prince}(x))$ where we instantiate x with William Rufus and y with William, we get prince(William Rufus).

From prince(William Rufus), ruthless(William Rufus) and $\forall x\, (\text{ruthless}(x) \wedge \text{prince}(x) \Rightarrow \text{king}(x))$, where x is instantiated to William Rufus, we deduce king(William Rufus).

Finally, from the last sentence $\forall x\, (\text{king}(x) \wedge (\text{crazy}(x) \vee \text{ruthless}(x)) \Rightarrow \neg\text{good}(x))$, with x instantiated to William Rufus, we can deduce that $\neg$good(William Rufus), since both king(Williams Rufus) and ruthless(William Rufus) hold.

Thus, we have proved both king(William Rufus) and $\neg$good(William Rufus), and the proposition king(William Rufus) $\wedge$ $\neg$good(William Rufus) follows from the knowledge base.

Marking:

- -2.5p if not clear how the first part "William Rufus was king" was proved

11. (5 points) **Logic: First-order logic—Task 3**

Consider the following knowledge base, given in natural language:

```
William was king.
All kings have at least one ruthless son.
All sons of kings are princes.
William Rufus and Henry were sons of William.
Henry was not ruthless, but William Rufus was.
All ruthless princes become kings.
No king who is ruthless or crazy is good.
```

Use the translation of this knowledge base into first-order logic that you did in the question "Logic: First-order logic—Task 1".

Convert the formalized knowledge base to CNF and divide into clauses. You must follow the algorithm for conversion given below (it was also given in the lecture and textbook). Document every step of your conversion into clause form.

1. Eliminate biconditionals and implications.

2. Move negation inwards, using the logical equivalences.
3. Standardize variables: each quantifier should use a different one
4. Skolemize: a more general form of existential instantiation.
5. Drop universal quantifiers.
6. Apply distributivity law (to distribute disjunction over conjunction).

Solution and Marking:

Here, only the final CNF clauses are shown:

1. king(William)
2. $\neg$king(x) $\vee$ son($f(x), x$)
   $\neg$king(x) $\vee$ ruthless($f(x)$)
3. $\neg$king(y) $\vee$ $\neg$son(x, y) $\vee$ prince(x)
4. son(William Rufus, William)
   son(Henry, William)
5. $\neg$ruthless(Henry)
   ruthless(William Rufus)
6. $\neg$ruthless(x) $\vee$ $\neg$prince(x) $\vee$ king(x)
7. $\neg$king(x) $\vee$ $\neg$crazy(x) $\vee$ $\neg$good(x)
   $\neg$king(x) $\vee$ $\neg$ruthless(x) $\vee$ $\neg$good(x)

Marking:

- 0.7p per right sentence

E Prolog (23 points)

12. (8 points) **Prolog: Constraint Logic Programming (CLP)**

Write a Prolog program that solves the cryptarithmetic puzzle below using CLP(FD). You need to find all concrete solutions.

```
    F A N
    F A N
+ S O F A
---------
H O U S E
```

Solution and Marking:

```
:- use_module(library(clpfd)).

puzzle([F,A,N] + [F,A,N] + [S,O,F,A] = [H,O,U,S,E]) :-
    Vars = [F,A,N,S,O,H,U,E],
    Vars ins 0..9,
    all_different(Vars),
    % arithmetic constraint that defines the puzzle
    F*100 + A*10 + N
 +  F*100 + A*10 + N
 +  S*1000 + O*100 + F*10 + A
 #=
    H*10000 + O*1000 + U*100 + S*10 + E,
    % letters start words, and hence cannot be zero
    F #\= 0, S #\= 0, H#\= 0,
    % attempt to ground as many variables as possible
    label(Vars).
```

Marking:

- -1 if `#=` is not used or incorrect
- -1 if `label(Vars)` is not used or 'var' instead of 'Vars', or extra commas in an expression

13. (7 points) **Prolog: Arithmetic operators**

Write a procedure `cubes_until(X, Y)` that simulates a *do-until* loop. The predicate shall write all cubes of numbers starting with `X` and increasing it by one until $X^3 > Y$. You must use **cut** (`!`) to exclude futile backtracking.

Example output of the procedure:

```
?- cubes_until(1, 50).
1
8
27
true.
?- cubes_until(2, 100).
8
27
64
true.
```

Solution and Marking:

```
% 1st rule - the condition to stop looping (the terminating case)
cubes_until(X, Y) :-
    %    Z is X ** 3,
    %    Z > Y,
```

```
    X**3 > Y, % '>' forces evaluation as well
    !.

% 2nd rule - the body of looping (the general case)
cubes_until(X, Y) :-
    Z is X ** 3,
    writeln(Z),
    X1 is X + 1,
    cubes_until(X1, Y).
```

Marking:

- 0p if no Prolog code, only pseudo code

14. (8 points) **Prolog: Data structures**

Define the predicate `in(X, Tree)`: to recognize whether `X` is in binary dictionary / binary search tree (BST) `Tree`.

A binary tree is written in the notation introduced in the lecture as well as in the textbook.

The notation is as follows:

- Let the atom `nil` represent the empty tree.
- Let the functor be `t` so the tree that has a root `X`, a left subtree `L` and a right subtree `R` is represented by the term `t(L, X, R)`.

To solve this problem, you need to recall what a binary dictionary / binary search tree (BST) is, and consider such trees consisting of only integer numbers. You must use **cut** (`!`) to improve the efficiency of the predicate.

Use this example BST from the lecture to test your solution:
`t( t(t(nil,1,nil), 3, t(nil,4,nil)), 5, t( t(nil, 6, t(nil,7,nil)), 8, t(nil,9,nil)) )`

Solution and Marking:

```
% in(X, Tree): X in binary dictionary / BST.
in(X, t( _, X, _) ) :- !.
in(X, t(Left, Root, Right) ) :-
    Root > X, !, % Root greater than X
    in(X, Left). % Search left subtree

in(X, t(Left, Root, Right) ) :-
    X > Root, % X greater than Root
    in(X, Right). % Search right subtree
```

Marking:

- 0p if no Prolog code, only pseudo code